BankQ — Project Execution Guide

Digital Queue Management System for Banks

Local Development • Standalone Execution • Production Deployments

1. Project Directory Overview

The project is structured into two completely independent, decoupled applications:

```
C:\Users\NIKHITHA\.gemini\antigravity-ide\scratch\digital-queue-management-system\
├── backend/    (Java 17, Spring Boot 3, Maven, H2 & PostgreSQL)
├── frontend/   (React 18, Vite 6, JavaScript, Axios REST Client)
├── RUN_GUIDE.md
└── README.md
```

2. Commands to Run Both Applications

Terminal 1: Start the Backend (Spring Boot Server)

Open PowerShell and navigate to the backend folder:

```
cd C:\Users\NIKHITHA\.gemini\antigravity-ide\scratch\digital-queue-management-system\backend
.\mvn.cmd spring-boot:run
```

> **Fast Alternative (Run the Pre-Packaged Standalone JAR directly):**
> `& "C:\Program Files\Microsoft\jdk-17.0.20.101-hotspot\bin\java.exe" -jar target\bankq-backend-1.0.0.jar`

- **Backend Base URL:** `http://localhost:8080`
- **Health Status Check:** `http://localhost:8080/api/health`
- **H2 Database Console:** `http://localhost:8080/h2-console`

Terminal 2: Start the Frontend (React + Vite Web App)

Open a second PowerShell window and navigate to the frontend folder:

```
cd C:\Users\NIKHITHA\.gemini\antigravity-ide\scratch\digital-queue-management-system\frontend
npm run dev
```

- **Frontend Website URL:** `http://localhost:5173`
- Requests to `/api` are automatically proxied to the backend on port 8080.

3. Authentication & Account Access

Method A: Sign Up with Your Own Credentials (Self Registration)

1. Open **http://localhost:5173** in your web browser.
2. Click the **"Create Account"** tab.

3. Fill in your **Full Name**, **Email Address**, and **Password** (min 6 characters).
4. Select your desired role:
   - **Customer:** Generate bank service tokens and track live queue progress.
   - **Staff:** Operate counter desks, call waiting customers, and complete tickets.
   - **Administrator:** Full system analytics, user directory, and master token registry.
5. Click **"Sign Up & Enter BankQ"** to immediately create your account and log in.

Method B: Pre-Seeded Demo Accounts

You can also use the one-click demo login buttons on the sign-in screen:

Role	Demo Email	Password	Capabilities
Customer	`customer@bankq.demo`	`Customer123`	Request tokens, view token history, track live queue
Staff	`staff@bankq.demo`	`Staff123`	Inspect waiting queue, call next customer, complete service
Administrator	`admin@bankq.demo`	`Admin123`	Executive dashboard, user list, queue load distribution

4. Inspecting the Database (H2 Console)

> **Note:** The file `bankqdb.mv.db` is an internal binary file. Do not open it in a text editor. Use the built-in browser console instead.

1. Open **http://localhost:8080/h2-console** in your browser.
2. Enter these connection values:
   - **Driver Class:** `org.h2.Driver`
   - **JDBC URL:** `jdbc:h2:file:./data/bankqdb`
   - **User Name:** `sa`
   - **Password:** *(leave blank)*
3. Click **Connect** to query tables: `USERS`, `QUEUE_TOKENS`, `BANKING_SERVICES`, and `COUNTERS`.

5. Production Build Commands

Package Backend (Standalone Executable JAR)

```
cd C:\Users\NIKHITHA\.gemini\antigravity-ide\scratch\digital-queue-management-system\backend
.\mvn.cmd package -DskipTests
```

Generates: `backend\target\bankq-backend-1.0.0.jar`

Build Frontend (Minified Static Bundle)

```
cd C:\Users\NIKHITHA\.gemini\antigravity-ide\scratch\digital-queue-management-system\frontend
npm run build
```

Generates: `frontend\dist\` (ready for Vercel, Netlify, or any static host)

6. Summary of Configured Banking Services

Code	Banking Service	Assigned Desk	Avg Duration
`CD`	Cash Deposit	Counter 1	5 mins

Code	Banking Service	Assigned Desk	Avg Duration
`CW`	Cash Withdrawal	Counter 2	5 mins
`AO`	Account Opening	Counter 3	15 mins
`PB`	Passbook Update	Counter 1	3 mins
`CH`	Cheque Services	Counter 2	8 mins
`LE`	Loan Enquiry	Counter 3	20 mins